

Tutorial on Hashing and Hash Functions

For Data Structure Students

Covers: hashing and hash functions, hash table structure and collisions, collision resolution techniques, maintaining load factor, and applications of hash tables.

Topic	Why it matters	Typical exam / lab focus
Hashing basics	Fast searching, insertion, and deletion	Design a good hash function
Collisions	Two keys may map to the same slot	Explain why collisions happen

Learning outcomes

- Define hashing and explain how a hash table stores data.
- Describe collisions and compare major collision resolution techniques.
- Compute and interpret the load factor of a hash table.
- Choose an appropriate hashing method for common applications.

1. Hashing and Hash Functions

Hashing is a technique used to convert a key into an index of an array. That array is called a hash table. Instead of scanning all stored elements one by one, we compute the position directly using a hash function. This is why hash tables often provide average-case $O(1)$ time for search, insertion, and deletion.

1.1 Basic idea

- A key is the value we want to store or search for, such as a roll number, username, or ID.
- A hash function $h(k)$ maps the key k to an integer index.
- The index tells us where the element should be placed in the table.

Example:

Suppose the table size is 10 and the hash function is $h(k) = k \bmod 10$. Then key 37 is stored at index 7, key 52 at index 2, and key 89 at index 9.

1.2 Properties of a good hash function

- It should be easy and fast to compute.
- It should distribute keys as uniformly as possible.
- It should minimize collisions.
- It should use all parts of the key rather than depending on only one digit or character.

Common hash functions

Method	Formula / idea	Suitable for
Division method	$h(k) = k \bmod m$	Integer keys; simple and common
Multiplication method	$h(k) = \text{floor}(m \times \text{frac}(kA))$	More uniform distribution
Mid-square method	Square the key and take middle digits	When digits vary well
Folding method	Split key into parts and add them	Long numeric keys
String hashing	Combine character codes systematically	Names, words, identifiers

1.3 Why hashing is useful

Hashing is most useful when fast access is more important than keeping data in sorted order. For example, checking whether a username already exists in a system is much faster with a hash table than with a simple linear search.

2. Hash Table Structure and Collisions

A hash table is usually implemented as an array of slots or buckets. Each slot stores one record, or a small collection of records, depending on the collision handling method.

2.1 Structure of a hash table

- Index range: 0 to $m-1$, where m is the table size.
- Each slot stores a key or a key-value pair.
- The index is obtained by applying the hash function.

Illustration:

For $h(k) = k \bmod 7$ and keys 10, 17, 24, 31:

0	1	2	3	4	5	6
			10, 17, 24, 31			

2.2 What is a collision?

A collision occurs when two or more different keys produce the same hash value. Collisions are unavoidable in practice because the number of possible keys is usually much larger than the number of table slots.

- If $h(10) = 3$ and $h(17) = 3$, then 10 and 17 collide.
- A collision does not mean hashing failed; it means we need a collision resolution method.

Important note for students:

A good hash function reduces collisions, but it cannot remove them completely when the table is finite.

3. Collision Resolution Techniques

When a collision occurs, the hash table must decide where to place the new key. The main techniques studied in data structures are separate chaining and open addressing.

3.1 Separate chaining

In separate chaining, each table slot stores a linked list or dynamic list of all keys that hash to that slot.

- Insert the new key into the list at the computed index.
- Search by going to the index and scanning only that list.
- Deletion is simpler than in many open-addressing methods.

Pros: simple, flexible, easy deletion. **Cons:** extra memory for pointers and list nodes.

3.2 Open addressing

In open addressing, all keys are stored inside the table itself. When a collision happens, we probe for another empty slot.

Technique	Probe rule	Example	Key point
Linear probing	$(h(k)+i) \bmod m$	Try next slot one by one	Simple but causes primary clustering
Quadratic probing	$(h(k)+i^2) \bmod m$	Jump by squares	Reduces clustering compared with linear
Double hashing	$(h_1(k) + i \cdot h_2(k)) \bmod m$	Second hash decides step size	Better distribution; often strongest option

3.3 Worked example: linear probing

Let table size $m = 7$ and $h(k) = k \bmod 7$. Insert keys 10, 17, 24, 31.

1. 10 → $h(10)=3$, place at index 3.
2. 17 → $h(17)=3$, collision. Check index 4, place there.
3. 24 → $h(24)=3$, collision. Check 4 occupied, check 5, place there.
4. 31 → $h(31)=3$, check 4 and 5 occupied, place at index 6.

3.4 Comparison summary

Method	Search	Deletion	Memory use
Separate chaining	Fast on average; depends on chain length	Easy	Extra memory needed
Linear probing	Fast when table is sparse	Needs care	Compact
Quadratic probing	Better than linear in many cases	Needs care	Compact
Double hashing	Usually very good	Needs care	Compact

4. Maintaining Load Factor

The load factor tells us how full the hash table is. It is one of the most important measures of performance.

Formula:

$$\text{Load factor } \alpha = n / m$$

Here, n is the number of stored elements and m is the total number of table slots.

Interpretation

- Small α means the table has many empty slots, so collisions are fewer.
- Large α means the table is crowded, so collisions increase.
- Performance generally gets worse as α increases.

Example:

If a table has 20 slots and currently stores 12 keys, then $\alpha = 12/20 = 0.6$.

How to maintain a healthy load factor

- Choose a suitable table size at the beginning, often a prime number.
- Monitor α during insertions.
- When α becomes too high, resize the table and rehash all elements.
- For many implementations, resizing is triggered around 0.7 to 0.75 in open addressing.

Rehashing

Rehashing means creating a larger table and inserting all existing keys again using the new table size. Although rehashing is expensive at that moment, it improves future performance.

5. Applications of Hash Tables

Hash tables are widely used because they support very fast lookup. They appear in both system software and everyday applications.

Application	Role of hashing
Compilers	Symbol tables store variable names, functions, and attributes.
Databases	Hash indexing speeds up exact-match search.
Caching	Recently used web pages or computed results can be found quickly.
Dictionaries / maps	Languages such as Python, Java, and C++ use hash-based structures.
Password storage	Cryptographic hashing is used to store passwords safely in transformed form.
Duplicate detection	Useful in sets, plagiarism checks, and data cleaning.

Important distinction

In data structures, a hash function is mainly used to map keys to slots efficiently. In security, cryptographic hash functions are designed for one-way transformation and integrity checking. Both use the term 'hash', but their goals are different.

6. Summary

- Hashing maps a key to an array index using a hash function.
- A hash table supports fast average-case insertion, deletion, and searching.
- A collision occurs when different keys map to the same slot.
- Collision resolution methods include separate chaining, linear probing, quadratic probing, and double hashing.
- Load factor $\alpha = n/m$ measures how full the table is.
- High load factor usually means more collisions and lower performance.
- Hash tables are used in symbol tables, dictionaries, caches, indexing, and many practical systems.

Practice Questions

5. Define hashing and explain the role of a hash function.
6. What is a collision? Why can collisions not be completely avoided?
7. Differentiate between separate chaining and open addressing.
8. Insert the keys 22, 32, 42, 52 into a table of size 10 using $h(k)=k \bmod 10$ and linear probing.
9. What is load factor? How does it affect performance?
10. List any three real-world applications of hash tables.